

PS5 Vault

Developer API Reference

Version 2.0 · API Version v1 · Built by Nookie

Base URL: `http://127.0.0.1:3731/api/v1`

Protocol: `HTTP REST + Server-Sent Events`

Auth: `None` — `localhost` only

Binding: `127.0.0.1` only (local machine)

Overview

PS5 Vault exposes a local HTTP API that lets external tools trigger scans, run transfers, and stream live progress events. The server binds exclusively to `127.0.0.1` — it is never reachable from outside the machine. Every request requires a 64-character hex API key.

The API starts automatically whenever PS5 Vault launches, in both normal and embedded modes. No additional configuration is required.

Note: Run headlessly with `PS5Vault.exe -embedded` to start without a UI window. A system-tray icon shows the process is running. The API behaves identically in both modes.

Conventions

Property	Value
Base URL	<code>http://127.0.0.1:3731/api/v1</code>
Port	3731 (fixed)
Content-Type	<code>application/json</code>
Max body	512 KB
CORS	Origin: * (all methods, all headers)

Authentication

Pass your API key in the `X-API-Key` header on every request.

```
X-API-Key: 8f3a2c1d...
```

Finding your key

The key is auto-generated on first run and saved to:

```
%APPDATA%\PS5 Vault\ps5vault-api-key.json
```

You can also copy it from **Menu** → **Developer API** inside the PS5 Vault UI, or regenerate it there. Regenerating immediately invalidates all existing integrations.

Error response for invalid key:

```
HTTP 401
```

```
{ "error": "Unauthorized: missing or invalid X-API-Key header" }
```

Endpoints

GET

/api/v1/status

App health + counts

Returns version, library size, and current scan/transfer states. Useful for health checks and polling loops.

Response

```
{
  "ok": true,
  "version": "2.4.2",
  "port": 3731,
  "library": { "count": 30 },
  "scan": { "active": false, "source": null, "progress": { "found": 0, "sized": 0, "total": 0 } },
  "transfer": { "active": false, "progress": {} }
}
```

GET

/api/v1/library

All scanned games

Returns every game in the current in-memory library. The library is populated by the last scan. Returns an empty array if no scan has run this session.

Response

```
{
  "count": 1,
  "games": [
    {
      "ppsa": "PPSA21564_00",
      "contentId": "UP9000-PPSA21564_00-000000000000000000",
      "title": "ASTRO BOT",
      "version": "01.007.000",
      "sdkVersion": "09000000",
    }
  ]
}
```

```

"region": "en-US",

"sizeBytes": 26843545600,

"sizeMb": 25600,

"folderPath": "T:\\Folder Games\\ASTRO BOT (01.007.000)",

"hasIcon": true,

"fwRequired": "09.00.00.00",

"titleId": "PPSA21564"

}

]

}

```

Game object fields

Field	Type	Description
ppsa	string null	PPSA identifier (e.g. PPSA21564_00)
contentId	string null	Full content ID from param.json
title	string null	Display title
version	string null	Content version (e.g. 01.007.000)
sdkVersion	string null	SDK version hex string
region	string null	Default language/region
sizeBytes	number null	Total size in bytes. null while sizing is in progress
sizeMb	number null	Size rounded to nearest MB
folderPath	string null	Absolute path to game folder on disk
hasIcon	boolean	Whether cover art is available via /icon endpoint
fwRequired	string null	Minimum firmware version string

titleId	string null	Title ID from param.json
---------	-------------	--------------------------

Note: sizeBytes may be null on large libraries while background sizing is in progress. Subscribe to the SSE stream for size-update events to receive sizes as they complete.

GET	/api/v1/library/:ppsa	Single game by PPSA ID
-----	-----------------------	------------------------

Retrieve a single game by PPSA ID. Lookup is case-insensitive. Returns the same shape as a single entry from /library.

```
GET /api/v1/library/PPSA21564_00
```

```
GET /api/v1/library/ppsa21564_00 # same result
```

Errors

Code	Body
404	Game not found: PPSA21564_00

GET	/api/v1/library/:ppsa/icon	Cover art PNG
-----	----------------------------	---------------

Returns the game's cover art as an image/png binary response. For locally scanned games this streams the file from disk. For FTP-scanned games it decodes the stored base64 blob.

Note: EventSource and plain tags do not support custom headers. Use fetch with createObjectURL instead.

```
const res = await fetch(
  'http://127.0.0.1:3731/api/v1/library/PPSA21564_00/icon',
  { headers: { 'X-API-Key': key } }
);

const blob = await res.blob();

img.src = URL.createObjectURL(blob);
```

Errors

Code	Body
404	Icon not found (game not found or no cover art)
500	Icon read error: [message]

POST	/api/v1/scan	Trigger a game scan
------	--------------	---------------------

Starts a scan asynchronously. Returns immediately — subscribe to /events for real-time progress, or poll /scan/status.

Request body

Field	Required	Notes
source	optional	Local path (D:\Games), FTP (192.168.1.100:2121), or "all-drives". Defaults to "all-drives" if omitted.

Examples

{ "source": "T:\\Folder Games" }
{ "source": "192.168.1.100:2121" }
{ "source": "all-drives" }

Response

{ "ok": true, "message": "Scan started", "source": "T:\\Folder Games" }

Errors

Code	Body
409	A scan is already running
400	Invalid JSON

GET	/api/v1/scan/status	Current scan state
-----	---------------------	--------------------

{
"active": true,
"source": "T:\\Folder Games",
"progress": { "found": 18, "sized": 6, "total": 18 }
}

active is false once both scanning and sizing are complete.

POST	/api/v1/transfer	Copy / move games
------	------------------	-------------------

Starts a transfer asynchronously. Returns immediately. Items must be full game objects from /library — pass them back as-is.

Request body

Field	Req.	Notes
items	YES	Array of game objects from /library. Must be non-empty.
dest	YES	Destination path. Local: E:\PS5. FTP: 192.168.1.100:2121.
action	no	copy (verified, default), copy-fast, move, folder-only
layout	no	Folder structure preset (see below). Default: etahen
overwriteMode	no	rename (default), overwrite, skip
customName	no	Custom folder name. Only used when layout is "custom".
ftpConfig	no	FTP config object for the source. Omit for local source.
ftpDestConfig	no	FTP config object for the destination. Omit for local dest.

Layout presets

Value	Output path structure
etahen	dest/etaHEN/games/Game Name (01.000.000)/
game-only	dest/Game Name (01.000.000)/
ppsa-only	dest/PPSA00000_00 (01.000.000)/
itemzflow	dest/games/Game Name (01.000.000)/
dump_runner	dest/homebrew/Game Name (01.000.000)/
porkfolio	dest/Game Name (01.000.000) PPSA00000_00/
game-ppsa	dest/Game Name (01.000.000)/PPSA00000_00/
custom	dest//

FTP config object

{
"host": "192.168.1.100",
"port": 2121,
"user": "anonymous",

```
"pass": "",  
"passive": true  
}
```

Full example

```
{  
  "items": [{ ...game object from /library... }],  
  "dest": "192.168.1.100:2121",  
  "action": "copy",  
  "layout": "etahen",  
  "overwriteMode": "rename",  
  "ftpDestConfig": {  
    "host": "192.168.1.100", "port": 2121,  
    "user": "anonymous", "pass": "", "passive": true  
  }  
}
```

Response

```
{ "ok": true, "message": "Transfer started", "itemCount": 1 }
```

Errors

Code	Body
400	items array is required / dest string is required / Invalid JSON
409	A transfer is already running

GET	/api/v1/transfer/status	Current transfer state
-----	-------------------------	------------------------

{
"active": true,
"progress": {
"type": "go-file-progress",
"itemIndex": 1,
"totalItems": 3,
"totalBytesCopied": 1073741824,
"totalBytes": 26843545600,
"grandTotalCopied": 1073741824,
"grandTotalBytes": 75161927680
}
}

GET	/api/v1/events	SSE live event stream
-----	----------------	-----------------------

Server-Sent Events stream. Connect once to receive all scan and transfer events in real time. The connection is kept alive with a ping comment every 25 seconds.

Note: EventSource does not support custom headers. Use fetch with a streaming reader as shown below.

Connecting

const res = await fetch('http://127.0.0.1:3731/api/v1/events',
{ headers: { 'X-API-Key': key } }
);
const reader = res.body
.pipeThrough(new TextDecoderStream())
.getReader();
while (true) {

```

const { done, value } = await reader.read();

if (done) break;

for (const line of value.split('\n')) {

  if (!line.startsWith('data:')) continue;

  const ev = JSON.parse(line.slice(5).trim());

  console.log(ev.type, ev.data);

}

}

```

Event envelope

```
data: { "type": "scan-start", "data": { ... }, "ts": 1741234567890 }
```

Event types

Type	Fired when	Key fields in data
connected	On connect	version, library (count)
scan-start	Scan begins	source
scan-progress	Each game found or sized	type (game-found size-update), item, done, total
size-update	Game size calculated	folderPath, totalSize
scan-complete	Scan + sizing done	count (total games)
scan-error	Scan threw an error	error (message)
transfer-start	Transfer begins	itemCount
transfer-progress	Per-file progress	type, itemIndex, totalItems, totalBytesCopied, grandTotalBytes
transfer-complete	Transfer finished	success: true
transfer-error	Transfer threw an error	error (message)

Code Examples

curl

```
# Status check

curl -H "X-API-Key: YOUR_KEY" http://127.0.0.1:3731/api/v1/status


# Trigger scan

curl -X POST -H "X-API-Key: YOUR_KEY" -H "Content-Type: application/json" \
-d '{"source":"T:\\Folder Games"}' \
http://127.0.0.1:3731/api/v1/scan


# List all game titles (requires jq)

curl -s -H "X-API-Key: YOUR_KEY" http://127.0.0.1:3731/api/v1/library \
| jq '.games[].title'


# Copy a game to PS5 over FTP

curl -X POST -H "X-API-Key: YOUR_KEY" -H "Content-Type: application/json" \
-d '{"items":[GAME_OBJECT],"dest":"192.168.1.100:2121","action":"copy","layout":"etahen",
"ftpDestConfig":{"host":"192.168.1.100","port":2121,"user":"anonymous","pass":"","passive":true}}' \
http://127.0.0.1:3731/api/v1/transfer
```

JavaScript

```
const BASE = 'http://127.0.0.1:3731/api/v1';

const KEY = 'YOUR_KEY';

const h = { 'X-API-Key': KEY, 'Content-Type': 'application/json' };

const api = (path, opts={}) => fetch(BASE+path, { headers:h, ...opts }).then(r=>r.json());


// 1. Trigger scan
```

```

await api('/scan', { method:'POST', body: JSON.stringify({ source: 'T:\\Folder Games' })
});

// 2. Poll until done

while (true) {

const s = await api('/scan/status');

if (!s.active) break;

await new Promise(r => setTimeout(r, 1000));

}

// 3. Find a game and copy to PS5

const { games } = await api('/library');

const game = games.find(g => g.title?.includes('Tekken 8'));

await api('/transfer', {

method: 'POST',

body: JSON.stringify({

items: [game], dest: '192.168.1.100:2121',

action: 'copy', layout: 'etahen',

ftpDestConfig: { host:'192.168.1.100', port:2121, user:'anonymous', pass:'', passive:true
}

}))

});

```

Python

```

import requests, time, json

BASE = 'http://127.0.0.1:3731/api/v1'

KEY = 'YOUR_KEY'

H = { 'X-API-Key': KEY, 'Content-Type': 'application/json' }

```

```
# Status
```

```
print(requests.get(f'{BASE}/status', headers=H).json())
```

```
# Scan
```

```
requests.post(f'{BASE}/scan', headers=H, json={ 'source': r'T:\Folder Games' })
```

```
# Poll until done
```

```
while True:
```

```
s = requests.get(f'{BASE}/scan/status', headers=H).json()
```

```
if not s['active']: break
```

```
time.sleep(1)
```

```
# Find game and copy
```

```
games = requests.get(f'{BASE}/library', headers=H).json()['games']
```

```
game = next(g for g in games if 'Tekken 8' in (g['title'] or ''))
```

```
requests.post(f'{BASE}/transfer', headers=H, json={
```

```
'items': [game], 'dest': r'E:\PS5Games',
```

```
'action': 'copy', 'layout': 'etahen'
```

```
})
```

```
# Stream SSE events
```

```
r = requests.get(f'{BASE}/events', headers=H, stream=True)
```

```
for line in r.iter_lines():
```

```
if line and line.startswith(b'data:')
```

```
ev = json.loads(line[5:])
```

```
print(ev['type'], ev.get('data', {}))
```

Error Reference

Status	Meaning
400	Bad request — missing required field, invalid JSON, or body too large (>512 KB)
401	Missing or invalid X-API-Key header
404	Unknown route, or game/icon not found
409	Conflict — a scan or transfer is already running
500	Internal server error — message in the error field

Embedded Mode

Start PS5 Vault with `--embedded` to run it as a background service with no UI window. The API server starts identically to normal mode.

```
PS5Vault.exe --embedded
```

A system-tray icon shows the app is running. Right-click it to open the full UI window or quit the process. This is the recommended mode for integration into game launchers, companion apps, or automation scripts.

Notes & Limitations

- **In-memory library.** The game library resets when PS5 Vault exits. Always trigger a scan before querying the library.
- **One operation at a time.** Only one scan and one transfer can run concurrently. Starting a second returns 409 Conflict.
- **Full game objects required.** Transfer items must be complete objects from `/library` — not just PPSA IDs.
- **sizeBytes may be null.** Background sizing continues after the scan completes. Subscribe to SSE size-update events for real-time updates.
- **Use fetch for SSE.** The EventSource API does not support custom headers. Use fetch with a streaming reader instead.
- **CORS is open by design.** Since the API only binds to 127.0.0.1, only code on the same machine can reach it regardless of CORS policy.

PS5 Vault Developer API · Built by Nookie